Previously, we saw one of the fundamental problems in concurrent programming: we would like to execute a series of instructions atomically but we couldn't due to interrupts.

Locks: The basic idea

- Assume example:

```
1  lock_t mutex; // some globally-allocated lock 'mutex'
2  ...
3  lock(&mutex);
4  balance = balance + 1;
5  unlock(&mutex);
```

- A lock is just a variable, which holds the state of the lock at any instant of time.
- It is either available (or unlocked or free) & thus no thread holds the lock or acquired (or locked or held) and thus exactly one thread holds the lock & is in a critical section.
- The lock variable can also store which thread holds the lock, or queue for ordering lock acquisition.

The semantics of the `lock()` and `unlock()` routines are simple. Calling the routine `lock()` tries to acquire the lock; if no other thread holds the lock (i.e., it is free), the thread will acquire the lock and enter the critical section; this thread is sometimes said to be the **owner** of the lock. If another thread then calls `lock()` on that same lock variable (`mutex` in this example), it will not return while the lock is held by another thread; in this way, other threads are prevented from entering the critical section while the first thread that holds the lock is in there.

Once the owner of the lock calls `unlock()`, the lock is now available (free) again. If no other threads are waiting for the lock (i.e., no other thread has called `lock()` and is stuck therein), the state of the lock is simply changed to free. If there are waiting threads (stuck in `lock()`), one of them will (eventually) notice (or be informed of) this change of the lock's state, acquire the lock, and enter the critical section.

Pthread Locks

The POSIX library uses for a lock is a mutex, as it is

used to provide mutual exclusion.

```
1  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
2
3  Pthread_mutex_lock(&lock); // wrapper; exits on failure
4  balance = balance + 1;
5  Pthread_mutex_unlock(&lock);
```

You might also notice here that the POSIX version passes a variable to lock and unlock, as we may be using *different* locks to protect different variables. Doing so can increase concurrency: instead of one big lock that is used any time any critical section is accessed (a **coarse-grained** locking strategy), one will often protect different data and data structures with different locks, thus allowing more threads to be in locked code at once (a more **fine-grained** approach).

Building A Lock

THE CRUX: HOW TO BUILD A LOCK

How can we build an efficient lock? Efficient locks provide mutual exclusion at low cost, and also might attain a few other properties we discuss below. What hardware support is needed? What OS support?

We will need help from hardware & OS to build locks.

Evaluating Locks

1. Correct: Whether the lock does its basic task, which is to provide mutual exclusion.

2. Fairness: Does each thread contending for the lock get a fair shot at acquiring it once it is free?

   OR

   Does any thread contending for the lock starve while doing so, thus never obtaining it?

3. Performance: The time overheads added by using lock.

   There are few cases to consider:

Case 1: When a single thread is running & grabs & releases the lock, what is the overhead of doing so?

Case 2: Where multiple threads are contending for the lock on a single CPU; in this case, are there performance concerns?

Case 3: How does the lock perform when multiple CPUs are involved, and threads on each contending for the lock?

Controlling Locks

One of the earliest solutions used to provide mutual exclusion was to disable interrupts for critical sections; this solution was invented for single processor

```
1  void lock() {
2      DisableInterrupts();
3  }
4  void unlock() {
5      EnableInterrupts();
6  }
```

+ Simplicity

- Requires us to allow to allow any calling thread to perform a privileged operation, and thus trust that this facility is not abused.

  - A greedy program would call lock() at beginning of its execution & thus monopolize the processor

  - A malicious program would call lock() & go into an endless loop & thus OS never regains control.

- This doesn't work on multi-processors. Interrupts are specific to CPUs, turning off on one CPU doesn't affect others.
  - For example, if multiple threads are running on different CPUs, and each try to enter the same critical section, it doesn't matter whether interrupts are disabled; threads will be able to run on other processors.

- Turning off interrupts for extended periods of time can lead to interrupts becoming lost.
  - For example, if the CPU missed the fact that a disk device has finished a read request. How will the OS know to wake the process waiting for read?

- Thus, only OS turns off interrupts for atomicity while updating its own data structures.

A failed attempt: Just using loads/stores

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 -> lock is available, 1 -> held
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1)  // TEST the flag
10         ; // spin-wait (do nothing)
11     mutex->flag = 1;          // now SET it!
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

In this first attempt, we use a simple variable (flag) to indicate whether some thread has possession of a lock. The first thread that enters the critical section will call lock(), which tests whether the flag is

equal to 1 (in this case, it is not), and then sets the flag to 1 to indicate that the thread now holds the lock. When finished with the critical section, the thread calls unlock() and clears the flag, thus indicating that the lock is no longer held. If another thread happens to call lock() while that first thread is in the critical section, it will simply spin-wait in the while loop for that thread to call unlock() and clear the flag. Once that first thread does so, the waiting thread will fall out of the while loop, set the flag to 1 for itself, and proceed into the critical section.

Correct X

Thread 1	Thread 2
call `lock()`	
while (flag == 1)	
interrupt: switch to Thread 2	
	call `lock()`
	while (flag == 1)
	flag = 1;
	interrupt: switch to Thread 1
flag = 1; // set flag to 1 (too!)	

Figure 28.2: **Trace: No Mutual Exclusion**

Performance X
↳ uses spin-wait
↳ CPU cycles wasted

Fairness X

Building Working Spin Locks with Test-And-Set

↳ We define test-and-set (or atomic exchange) via C code snippet:

```
1  int TestAndSet(int *old_ptr, int new) {
2      int old = *old_ptr; // fetch old value at old_ptr
3      *old_ptr = new;     // store 'new' into old_ptr
4      return old;         // return the old value
5  }
```

↳ It returns the old value pointed to by old_pointer, and simultaneously updates said value to new.

This sequence of operations is performed atomically by the hardware.

↳ It is called TestAndSet because it allows you to "test" the old value while "setting" the memory location to new value.

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0: lock is available, 1: lock is held
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // spin-wait (do nothing)
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }
```

Let's make sure we understand why this lock works. Imagine first the case where a thread calls `lock()` and no other thread currently holds the lock; thus, `flag` should be 0. When the thread calls `TestAndSet(flag, 1)`, the routine will return the old value of `flag`, which is 0; thus, the calling thread, which is *testing* the value of flag, will not get caught spinning in the while loop and will acquire the lock. The thread will also atomically *set* the value to 1, thus indicating that the lock is now held. When the thread is finished with its critical section, it calls `unlock()` to set the flag back to zero.

The second case we can imagine arises when one thread already has the lock held (i.e., `flag` is 1). In this case, this thread will call `lock()` and then call `TestAndSet(flag, 1)` as well. This time, `TestAndSet()` will return the old value at flag, which is 1 (because the lock is held), while simultaneously setting it to 1 again. As long as the lock is held by another thread, `TestAndSet()` will repeatedly return 1, and thus this thread will spin and spin until the lock is finally released. When the flag is finally set to 0 by some other thread, this thread will call `TestAndSet()` again, which will now return 0 while atomically setting the value to 1 and thus acquire the lock and enter the critical section.

Evaluating Spin Locks

This is the same evaluation for Compare-And-Swap/Load-Linked

Correct ✓

Fairness ✗ Don't provide any guarantee for fairness.

Performance ✗

- Brutal for single processor
- Bearable for multi-processor especially if critical section is short.

ASIDE: DEKKER'S AND PETERSON'S ALGORITHMS

In the 1960's, Dijkstra posed the concurrency problem to his friends, and one of them, a mathematician named Theodorus Jozef Dekker, came up with a solution [D68]. Unlike the solutions we discuss here, which use special hardware instructions and even OS support, **Dekker's algorithm** uses just loads and stores (assuming they are atomic with respect to each other, which was true on early hardware).

Dekker's approach was later refined by Peterson [P81]. Once again, just loads and stores are used, and the idea is to ensure that two threads never enter a critical section at the same time. Here is **Peterson's algorithm** (for two threads); see if you can understand the code. What are the `flag` and `turn` variables used for?

```
int flag[2];
int turn;

void init() {
    // indicate you intend to hold the lock w/ 'flag'
    flag[0] = flag[1] = 0;
    // whose turn is it? (thread 0 or 1)
    turn = 0;
}
void lock() {
    // 'self' is the thread ID of caller
    flag[self] = 1;
    // make it other thread's turn
    turn = 1 - self;
    while ((flag[1-self] == 1) && (turn == 1 - self))
        ; // spin-wait while it's not your turn
}
void unlock() {
    // simply undo your intent
    flag[self] = 0;
}
```

For some reason, developing locks that work without special hardware support became all the rage for a while, giving theory-types a lot of problems to work on. Of course, this line of work became quite useless when people realized it is much easier to assume a little hardware support (and indeed that support had been around from the earliest days of multiprocessing). Further, algorithms like the ones above don't work on modern hardware (due to relaxed memory consistency models), thus making them even less useful than they were before. Yet more research relegated to the dustbin of history...

Compare-And-Swap

```
1  int CompareAndSwap(int *ptr, int expected, int new) {
2      int original = *ptr;
3      if (original == expected)
4          *ptr = new;
5      return original;
6  }
```

Test if value at the address specified by ptr is equal to expected; if so, update the memory location pointed to by ptr with the new value.

If not, do nothing.

```
1  void lock(lock_t *lock) {
2      while (CompareAndSwap(&lock->flag, 0, 1) == 1)
3          ; // spin
4  }
```

Load-Linked & Store-Conditional

- Load-Linked simply fetches a value from memory & places it in a register.
- Store-Conditional succeeds (and updates the value stored at the address just load-linked from) if no intervening store to the address has taken place.

  In case of success, store-conditional returns 1, and updates value at ptr to value.

  If it fails, the value at ptr is not updated & 0 is returned.

The `lock()` code is the only interesting piece. First, a thread spins waiting for the flag to be set to 0 (and thus indicate the lock is not held). Once so, the thread tries to acquire the lock via the store-conditional; if it succeeds, the thread has atomically changed the flag's value to 1 and thus can proceed into the critical section.

Note how failure of the store-conditional might arise. One thread calls `lock()` and executes the load-linked, returning 0 as the lock is not held. Before it can attempt the store-conditional, it is interrupted and another thread enters the lock code, also executing the load-linked instruction, and also getting a 0 and continuing. At this point, two threads have each executed the load-linked and each are about to attempt the store-conditional. The key feature of these instructions is that only one of these threads will succeed in updating the flag to 1 and thus acquire the lock; the second thread to attempt the store-conditional will fail (because the other thread updated the value of flag between its load-linked and store-conditional) and thus have to try to acquire the lock again.

```
1  int LoadLinked(int *ptr) {
2      return *ptr;
3  }
4
5  int StoreConditional(int *ptr, int value) {
6      if (no update to *ptr since LL to this addr) {
7          *ptr = value;
8          return 1; // success!
9      } else {
10         return 0; // failed to update
11     }
12 }
```

```
1  void lock(lock_t *lock) {
2      while (1) {
3          while (LoadLinked(&lock->flag) == 1)
4              ; // spin until it's zero
5          if (StoreConditional(&lock->flag, 1) == 1)
6              return; // if set-to-1 was success: done
7                      // otherwise: try again
8      }
9  }
10
11 void unlock(lock_t *lock) {
12     lock->flag = 0;
13 }
```

Or concisely,

```
1  void lock(lock_t *lock) {
2      while (LoadLinked(&lock->flag) ||
3             !StoreConditional(&lock->flag, 1))
4          ; // spin
5  }
```

Fetch-And-Add

- Atomically increments a value while returning old value at a particular address

```
1   int FetchAndAdd(int *ptr) {
2       int old = *ptr;
3       *ptr = old + 1;
4       return old;
5   }
```

- We use this to build ticket locks

- When a thread wishes to acquire a lock, it first does an atomic fetch-and-add on ticket value.

  That value is considered this thread's "turn" (`myturn`).

  The global `lock->turn` is used to determine which thread's turn it is; when `(myturn == turn)`

- Unlock is accomplished by incrementing the turn.

Correct ✓

Fairness ✓: Once a thread is assigned a ticket value, it will be scheduled at some point in future.

Performance ✗

```
1   typedef struct __lock_t {
2       int ticket;
3       int turn;
4   } lock_t;
5
6   void lock_init(lock_t *lock) {
7       lock->ticket = 0;
8       lock->turn   = 0;
9   }
```

```
10
11  void lock(lock_t *lock) {
12      int myturn = FetchAndAdd(&lock->ticket);
13      while (lock->turn != myturn)
14          ; // spin
15   }
16
17  void unlock(lock_t *lock) {
18      lock->turn = lock->turn + 1;
19  }
```

Figure 28.7: Ticket Locks

Too much spinning: What Now?

thread (thread 0) is in a critical section and thus has a lock held, and unfortunately gets interrupted. The second thread (thread 1) now tries to acquire the lock, but finds that it is held. Thus, it begins to spin. And spin. Then it spins some more. And finally, a timer interrupt goes off, thread 0 is run again, which releases the lock, and finally (the next time it runs, say), thread 1 won't have to spin so much and will be able to acquire the lock. Thus, any time a thread gets caught spinning in a situation like this, it wastes an entire time slice doing nothing but checking a value that isn't going to change! The problem gets worse with N threads contending for a lock; $N - 1$ time slices may be wasted in a similar manner, simply spinning and waiting for a single thread to release the lock. And thus, our next problem:

Now imagine that one

THE CRUX: HOW TO AVOID SPINNING

How can we develop a lock that doesn't needlessly waste time spinning on the CPU?

A Simple Approach: Just Yield, Zayan Baby

```
1   void init() {
2       flag = 0;
3   }
4
5   void lock() {
6       while (TestAndSet(&flag, 1) == 1)
7           yield(); // give up the CPU
8   }
9
```

```
10  void unlock() {
11      flag = 0;
12  }
```

↳ yield() is a system call that moves the caller from the running state to ready state. Thus, yielding thread de-schedules itself.

Think about the example with two threads on one CPU; in this case, our yield-based approach works quite well. If a thread happens to call `lock()` and find a lock held, it will simply yield the CPU, and thus the other thread will run and finish its critical section. In this simple case, the yielding approach works well.

Let us now consider the case where there are many threads (say 100) contending for a lock repeatedly. In this case, if one thread acquires the lock and is preempted before releasing it, the other 99 will each call `lock()`, find the lock held, and yield the CPU. Assuming some kind of round-robin scheduler, each of the 99 will execute this run-and-yield pattern before the thread holding the lock gets to run again. While better than our spinning approach (which would waste 99 time slices spinning), this approach is still costly; the cost of a context switch can be substantial, and there is thus plenty of waste.

Using Queues: Sleeping Instead of Spinning

↳ Except ticket lock, no previous approach ensures fairness

↳ So, we need to explicitly exert control over which thread next gets to acquire the lock after current holder releases it.

park(): puts a thread to sleep.

unpark(threadID): wake a particular thread as designated by threadID.

```
1   typedef struct __lock_t {
2       int flag;
3       int guard;
4       queue_t *q;
5   } lock_t;
6
7   void lock_init(lock_t *m) {
8       m->flag  = 0;
9       m->guard = 0;
10      queue_init(m->q);
11  }
```

```
12
13  void lock(lock_t *m) {
14      while (TestAndSet(&m->guard, 1) == 1)
15          ; //acquire guard lock by spinning
16      if (m->flag == 0) {
17          m->flag = 1; // lock is acquired
18          m->guard = 0;
19      } else {
20          queue_add(m->q, gettid());
21          m->guard = 0;
22          park();
23      }
24  }
25
26  void unlock(lock_t *m) {
27      while (TestAndSet(&m->guard, 1) == 1)
28          ; //acquire guard lock by spinning
29      if (queue_empty(m->q))
30          m->flag = 0; // let go of lock; no one wants it
31      else
32          unpark(queue_remove(m->q)); // hold lock
33                                      // (for next thread!)
34      m->guard = 0;
35  }
```

→ This ensures atomicity

This needs to be atomic

Observations:

1. Guard is used as a spin-lock around flag & queue manipulations.

   This approach thus doesn't entirely avoid spin-waiting entirely; a thread might be interrupted while acquiring or releasing the lock, and thus other threads spin-wait for this one.

   However, the time spent spinning is quite limited.

2. When a thread cannot acquire the lock, we add that thread to a queue, set guard to 0, and yield the CPU.

3. flag is NOT set to 0, when another thread gets woken up.

   When a thread is woken up, it will be as if it is returning from park(); however, it doesn't hold guard at that point in the code & thus cannot even try to set the flag to 1.

Thus, we just pass lock directly from thread releasing the lock to next thread; flag is not set to 0 in-between.

4. There is a race condition. A thread that is about to park is a switch at that time to the thread holding the lock who just releases the lock.

The subsequent park() by the first thread would sleep forever.

Solaris solves this by adding setpark(), a thread can indicate it is about to park().

```
1        queue_add(m->q, gettid());
2        setpark(); // new code
3        m->guard = 0;
```

Different OS, Different Support

- Linux provides a futex. Each futex has associated with it a specific physical memory locations, as well as a per-futex in-kernel queue.

- futex_wait(address, expected) puts the calling thread to sleep, assuming the value at the address `address` is equal to `expected`.

- futex_wake(address) wakes one thread that is waiting in the queue.

- It uses a single integer to indicate whether the lock is held or not (high bit)

Thus, if a lock is negative, it is held (because high bit is set)

```
1  void mutex_lock (int *mutex) {
2    int v;
3    // Bit 31 was clear, we got the mutex (fastpath)
4    if (atomic_bit_test_set (mutex, 31) == 0)
5      return;
6    atomic_increment (mutex);
7    while (1) {
8        if (atomic_bit_test_set (mutex, 31) == 0) {
9            atomic_decrement (mutex);
10           return;
11       }
12       // Have to waitFirst to make sure futex value
13       // we are monitoring is negative (locked).
14       v = *mutex;
15       if (v >= 0)
16         continue;
17         futex_wait (mutex, v);
18   }
19 }
20
21 void mutex_unlock (int *mutex) {
22   // Adding 0x80000000 to counter results in 0 if and
23   // only if there are not other interested threads
24   if (atomic_add_zero (mutex, 0x80000000))
25     return;
26
27   // There are other threads waiting for this mutex,
28   // wake one of them up.
29   futex_wake (mutex);
30 }
```

Two-Phase Locks

A two-phase lock realizes that spinning can be useful, particularly if lock is about to be released.

So, in first phase, the lock spins for a while, hoping that it can acquire the lock.

However, if lock is not acquired during phase 1, 2nd phase is entered, where caller is put to sleep.